

Avances Proyecto

A nivel de código, implementamos el Patrón Observador. En lugar de que código dependiera unos de otros, se usó `Action` de C#

```
3 references
public event Action OnAtaqueSolicitado;

Unity Message | 0 references
void Start()
{
    controlArma = GetComponent<AgarrarArma>();
}

Unity Message | 0 references
void Update()
{
    if (controlArma != null && controlArma.tieneArma)
    {
        return;
    }

    if (Time.time >= tiempoSiguienteAtaque)
    {
        if (Input.GetButtonDown("Fire1"))
        {
            Golpear();
            tiempoSiguienteAtaque = Time.time + tiempoEntreAtaques;
        }
    }
}
```

Este descubrimiento nos permitió separar `JugadorAtaque`, `SaludEnemigo` y `ControlDeAnimacionesJugador`. Cuando el jugador ataca, el código emite la señal `OnAtaqueRealizado`. El `Animator` la escucha y reproduce la animación sin que los códigos estén acoplados.

Aspectos Técnicos:

Hubo un problema con que la Linterna no podía apuntar porque el código estaba condicionado al TipoArma. Después de pensarlo, implementamos un sistema basado en Booleans, haciendo tus objetos modulares.

```
using UnityEngine;

2 references
public enum TipoArma
{
    0 references
    DeFuego,
    1 reference
    CuerpoACuerpo,
    0 references
    Herramienta
}

⊞ Unity Script (3 asset references) | 5 references
public class DatosArma : MonoBehaviour
{
    ⊞ Unity Serialized Field (multiple values) | 1 reference
    public TipoArma tipoArma;

    [Header("Capacidades del Objeto")]
    ⊞ Unity Serialized Field | 1 reference
    public bool puedeApuntar = true; // Si es true, la manos se levanta
    ⊞ Unity Serialized Field | 1 reference
    public bool puedeDisparar = false; // Si es true, usa munición y crea proyectiles
    ⊞ Unity Serialized Field | 0 references
    public bool puedeAtacarMelee = false; // Si es true, realiza un golpe cuerpo a cuerpo

    [Header("Configuración de Linterna (Opcional)")]
    ⊞ Unity Serialized Field | 3 references
    public Light componenteLuz;

    [Header("Estadísticas (Si aplica)")]
    ⊞ Unity Serialized Field (multiple values) | 1 reference
    public int dano = 10;
    ⊞ Unity Serialized Field (multiple values) | 1 reference
    public float rangoAtaque = 2f;
}
```

En lugar de crear un script para pistolas, otro para armas cuerpo a cuerpo y otro para linternas, ahora en DatosArma es todo universal. El comportamiento del jugador ya no depende de "qué es" el objeto, sino de "qué puede hacer". Esto también ayuda porque si en un futuro queremos agregar más armas u objetos, ya no habrá que especificar el comportamiento de cada uno.

Integración Cross-Platform: VR y PC

Migramos la detección de clics tradicionales de Input.GetMouse al nuevo Input System de Unity mediante InputActionReference. Esto es para recibir entradas tanto del ratón de una PC como del Grip de un mando VR sin alterar una sola línea de código.

```
⊖ Unity Serialized Field | 6 references  
public InputActionReference accionDisparar;  
⊖ Unity Serialized Field | 6 references  
public InputActionReference accionApuntar;  
⊖ Unity Serialized Field | 6 references  
public InputActionReference accionRecargar;
```

```
// Apuntar  
if (armaActual.puedeApuntar)  
{  
    if (accionApuntar != null && accionApuntar.action.IsPressed())  
    {  
        // ...  
    }  
}
```

Vimos que declarar la variable no era suficiente ya que el Input System requiere el método `OnEnable` para "encender" la escucha de los botones. Al combinar esto con `armaActual.puedeApuntar`, se logró un control universal de mandos.